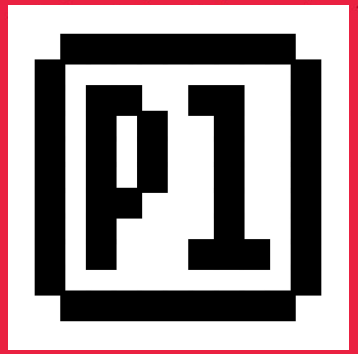
GBD+ vscode+ Doxygen



Mauro Fermín
mafermin@unrn.edu.ar

UNRN
Universidad Nacional
de **Río Negro**

r33

06 / 2026

ASSERT

Sintaxis

```
#include <assert.h>

assert(condicion);
```

- Si la condición es **verdadera** → el programa continúa normalmente
- Si la condición es **falsa** → el programa se detiene y muestra el error

Ejemplos - assert

Ejemplo 1: Condición básica

```
int edad = 20;
assert(edad >= 0);
```

"La edad nunca debería ser negativa."

Ejemplo 2: Precondición

```
float dividir(float a, float b)
{
  assert(b != 0);
  return a / b;
}
```

El divisor debe ser diferente de cero.

Ejemplo 3: Verificar resultado

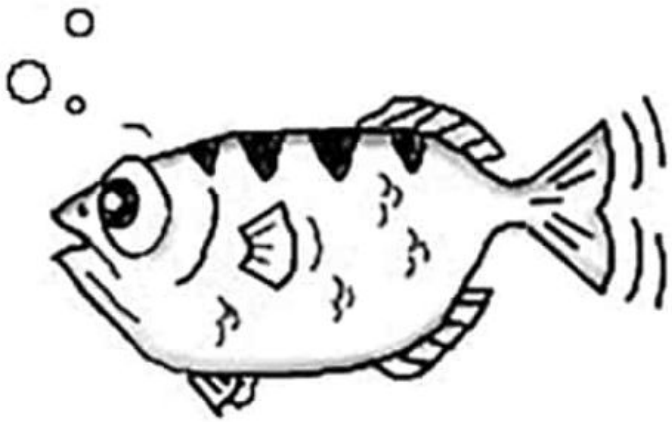
```
int cuadrado(int x)
{
  int resultado = x * x;
  assert(resultado >= 0);
  return resultado;
}
```

El resultado debería cumplir esta condición.

Ejemplo 4: Verificar función

```
assert(es_par(numero));
```

numero debería ser par en este punto.



GDB

The GNU Project Debugger

Comandos principales

`gdb ./programa`

Inicia GDB

`run / r`

Ejecuta el programa

`continue / c`

Continúa hasta el próximo breakpoint

`print x / p x`

Muestra el valor de x

`break main / b main`

Coloca un breakpoint en main

`break 20`

Breakpoint en línea 20

`info locals`

Muestra variables locales

`backtrace / bt`

Muestra la pila de llamadas

`next / n`

Ejecuta la siguiente línea

`step / s`

Entra dentro de una función

`list / l`

Muestra código fuente

`quit / q`

Sale de GDB

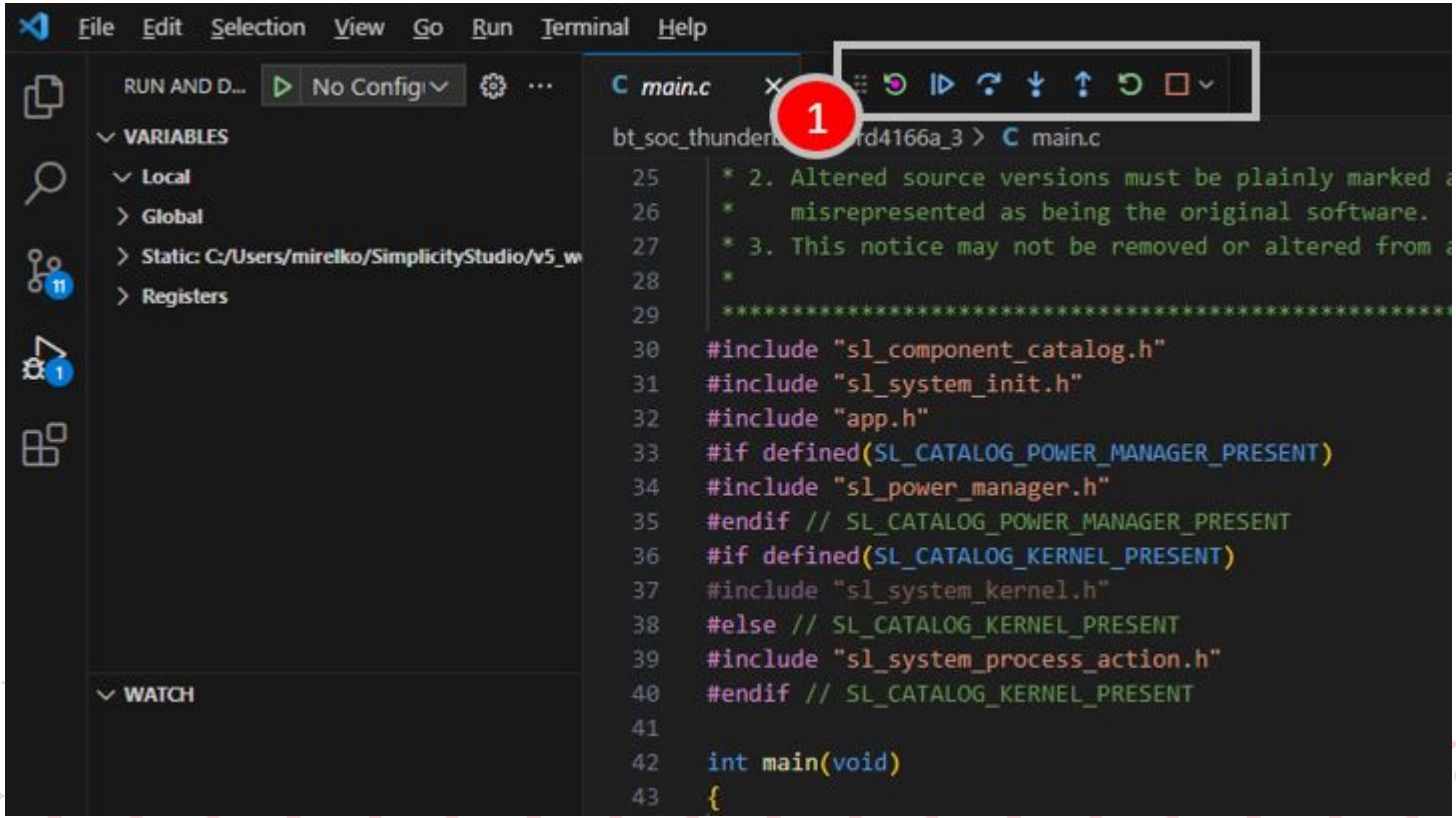
Compilar para debugging

```
gcc -Wall -Wextra -g main.c -o programa
```

`-Wall` → activa conjunto de advertencias comunes.
`-Wextra` → activa advertencias adicionales que `-Wall` no incluye.

```
int x;

printf("%d\n", 10);
```


File Edit Selection View Go Run Terminal Help
RUN AND D...
No Config
main.c
1
VARIABLES
Local
Global
Static: C:/Users/mirelko/SimplicityStudio/v5_w
Registers
WATCH
bt_soc_thunder
rd4166a_3 > C main.c
25 * 2. Altered source versions must be plainly marked a
26 * misrepresented as being the original software.
27 * 3. This notice may not be removed or altered from a
28 *
29 **
30 #include "sl_component_catalog.h"
31 #include "sl_system_init.h"
32 #include "app.h"
33 #if defined(SL_CATALOG_POWER_MANAGER_PRESENT)
34 #include "sl_power_manager.h"
35 #endif // SL_CATALOG_POWER_MANAGER_PRESENT
36 #if defined(SL_CATALOG_KERNEL_PRESENT)
37 #include "sl_system_kernel.h"
38 #else // SL_CATALOG_KERNEL_PRESENT
39 #include "sl_system_process_action.h"
40 #endif // SL_CATALOG_KERNEL_PRESENT
41
42 int main(void)
43 {


doxygen

¿Qué es Doxygen?

- **Herramienta de generación de documentación automática**: Agiliza el proceso de creación de manuales y referencias a partir del código fuente.
- **Extrae comentarios especiales del código fuente**: Reconoce sintaxis específicas para integrar descripciones, parámetros y ejemplos directamente de tu código.
- **Genera documentación en HTML, LaTeX, RTF y otros formatos**: Ofrece versatilidad para publicar la documentación en diversos medios, adaptándose a las necesidades del equipo.
- **Soporta múltiples lenguajes incluyendo C, C++, Java, Python**: Amplia compatibilidad con los lenguajes de programación más utilizados, haciendo de Doxygen una solución robusta.
- **Ideal para proyectos grandes y colaborativos**: Facilita el mantenimiento de la documentación actualizada en entornos con múltiples desarrolladores.

Sintaxis

Para que Doxygen pueda generar documentación efectiva a partir de su código C, es crucial utilizar un formato de comentario específico. Los bloques de comentarios de Doxygen para funciones y estructuras principales deben comenzar con `/**` y terminar con `*/`. Esto indica a la herramienta que el contenido dentro de estos delimitadores debe ser procesado para generar la documentación.

A continuación, se presentan algunas de las etiquetas más importantes utilizadas dentro de estos comentarios:

- `@brief`: Proporciona una descripción concisa de una línea para el elemento.
- `@param <nombre_parametro>`: Describe un parámetro específico de la función, detallando su propósito.
- `@return`: Explica el valor que la función devuelve.
- `@note`: Se utiliza para añadir una nota importante o una advertencia relevante.

Ejemplo:

```
/**
 * @brief Calcula la suma de dos números enteros.
 *
 * Esta función toma dos números enteros como entrada y devuelve
 * su suma. Es un ejemplo simple para demostrar la sintaxis de Doxygen.
 *
 * @param a El primer número entero a sumar.
 * @param b El segundo número entero a sumar.
 * @return La suma de 'a' y 'b'.
 * @note Se asume que la suma de 'a' y 'b' no excederá el rango
 *       de un entero estándar para evitar desbordamientos.
 */
int sumar(int a, int b) {
    return a + b;
}
```

Configuración recomendada

```
PROJECT_NAME       = "Progra I"
OUTPUT_DIRECTORY   = doc

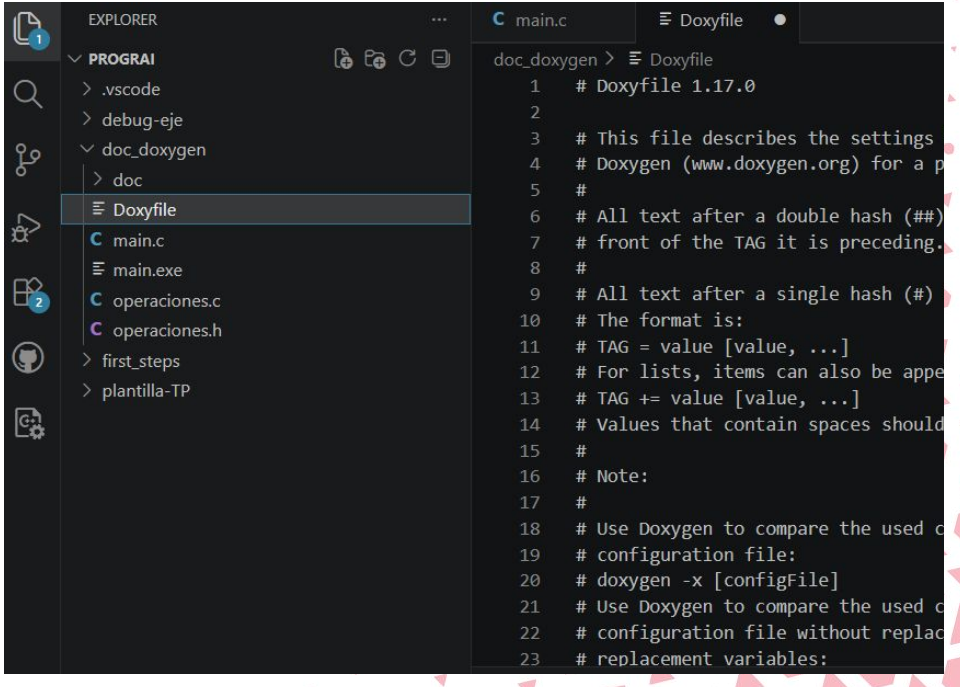
INPUT              = .
RECURSIVE          = YES
FILE_PATTERNS      = *.c *.h

EXTRACT_ALL        = YES
EXTRACT_PRIVATE    = NO

GENERATE_HTML      = YES
HTML_OUTPUT        = html

GENERATE_LATEX     = NO

WARNINGS           = YES
WARN_IF_UNDOCUMENTED  = YES
WARN_IF_DOC_ERROR  = YES
```



Comandos >> bash

1. Verificar la versión de Doxygen

Este comando te permite confirmar que Doxygen está correctamente instalado en tu sistema y cuál es la versión que tienes disponible.

```
> doxygen --version
```

2. Generar el archivo de configuración Doxyfile

El Doxyfile es el corazón de tu configuración de Doxygen. Este comando crea un archivo de configuración por defecto que puedes personalizar según el proyecto.

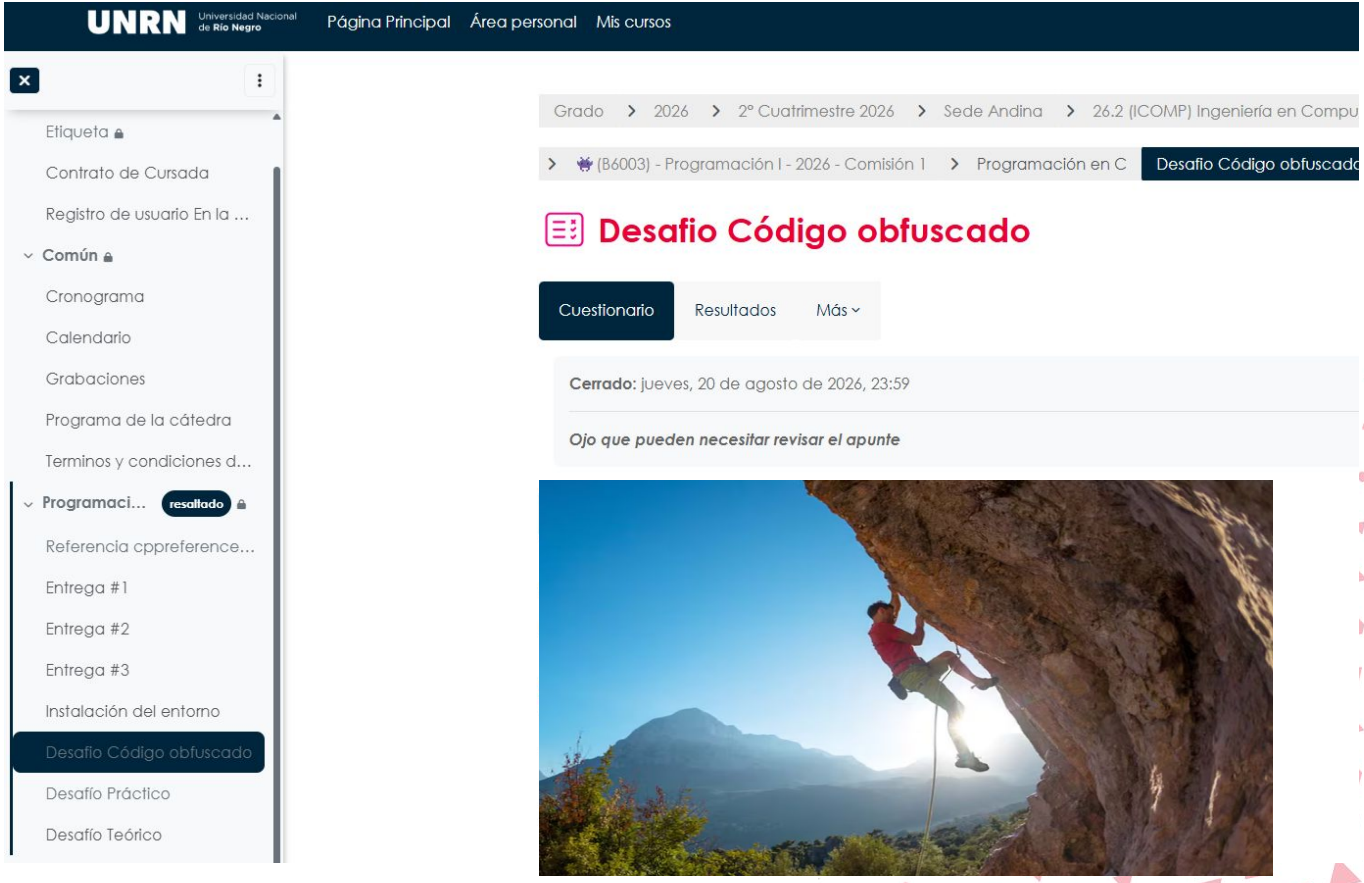
```
> doxygen -g
```

3. Ejecutar Doxygen con el archivo de configuración

Una vez que tienes tu Doxyfile configurado, este comando procesa tus archivos fuente y genera la documentación en el formato especificado (**HTML**, LaTeX, etc.).

```
>doxygen Doxyfile
```

DESAFÍO



UNRN | Universidad Nacional de Río Negro

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar